



LINGUAGENS DE PROGRAMAÇÃO PARA CIÊNCIA DE DADOS (PYTHON COM SPARK)

Rodrigo Amaral Rocha

LINGUAGENS DE PROGRAMAÇÃO PARA CIÊNCIA DE DADOS (PYTHON COM SPARK)

1ª edição

Londrina
Editora e Distribuidora Educacional S.A.
2024

© 2024 por Editora e Distribuidora Educacional S.A.

Todos os direitos reservados. Nenhuma parte desta publicação poderá ser reproduzida ou transmitida de qualquer modo ou por qualquer outro meio, eletrônico ou mecânico, incluindo fotocópia, gravação ou qualquer outro tipo de sistema de armazenamento e transmissão de informação, sem prévia autorização, por escrito, da Editora e Distribuidora Educacional S.A.

Diretora Sr. de Pós-graduação & OPM

Silvia Rodrigues Cima Bizatto

Conselho Acadêmico

Alessandra Cristina Fahl

Ana Carolina Gulelmo Staut

Camila Braga de Oliveira Higa

Camila Turchetti Bacan Gabiatti

Giani Vendramel de Oliveira

Gislaine Denisale Ferreira

Henrique Salustiano Silva

Juliana Schiavetto Dauricio

Juliane Raniro Hehl

Mariana Gerardi Mello

Nirse Ruscheinsky Breternitz

Coordenador

Juliana Dauricio Schiavetto

Revisor

Elisa Antolli

Editorial

Beatriz Meloni Montefusco

Márcia Regina Silva

Paola Andressa Machado Leal

Rosana Silverio Siqueira

Dados Internacionais de Catalogação na Publicação (CIP)

R672l Rocha, Rodrigo Amaral
Linguagens de programação para ciência de dados (Python com Spark) /Rodrigo Amaral Rocha, – Londrina: Editora e Distribuidora Educacional S.A., 2024.
32 p.

ISBN 978-65-5903-668-4

1. Apache Spark. 2. Pandas. 3. NumPy. I. Título.

CDU 004.4

Raquel Torres – CRB 8/10534

2024

Editora e Distribuidora Educacional S.A.
Avenida Paris, 675 – Parque Residencial João Piza
CEP: 86041-100 — Londrina — PR
Homepage: <https://www.cogna.com.br/>



LINGUAGENS DE PROGRAMAÇÃO PARA CIÊNCIA DE DADOS (PYTHON COM SPARK)

SUMÁRIO

Apresentação da disciplina	05
Linguagens de programação para ciência de dados: configuração de ambiente, sintaxe e bibliotecas	07
Big Data e Apache Spark	22
PySpark – Análise de Dados	34
Aplicações Avançadas e Visualização de Dados	55

Apresentação da disciplina

Bem-vindo(a) à *disciplina Linguagens de Programação para Ciência de Dados (Python com Spark)*!

Esta disciplina é essencial para qualquer profissional que queira se destacar na análise e processamento de grandes volumes de dados. O enfoque principal está em Python e Apache Spark, duas ferramentas poderosas e amplamente usadas na indústria para manipulação e análise de dados. Python, com suas bibliotecas como Pandas, NumPy, Matplotlib e SciPy, oferece uma plataforma sólida para análise de dados, enquanto o Apache Spark permite o processamento distribuído eficiente de conjuntos extensos de dados.

Durante o curso, você desenvolverá uma compreensão dos conceitos básicos da programação em Python, algo fundamental para qualquer cientista de dados. Você aprenderá a lidar com dados em diversos formatos, uma habilidade de grande importância em um mundo em que os dados provêm de várias fontes e têm diferentes formas. A capacidade de manipular e processar grandes volumes de dados com Apache Spark será uma vantagem considerável, permitindo que você trabalhe com Big Data de forma eficaz.

Além disso, a matéria irá orientá-lo no uso das bibliotecas e frameworks do Python para análise de dados, capacitando-o a implementar algoritmos de aprendizado de máquina e executar processamento distribuído usando PySpark. Essas habilidades não apenas aumentarão seu valor no mercado de trabalho, mas também fornecerão ferramentas práticas para resolver problemas complexos e tomar decisões fundamentadas em dados.



Ao término desta disciplina, você estará preparado para enfrentar desafios reais relacionados a Big data, aplicando técnicas avançadas de análise de dados altamente valorizadas no mercado. Seja você um futuro cientista de dados, analista ou engenheiro de dados, as habilidades que adquirirá aqui são fundamentais para o seu desenvolvimento profissional.

Linguagens de programação para ciência de dados: configuração de ambiente, sintaxe e bibliotecas

Autoria: Rodrigo Amaral Rocha

Leitura crítica: Elisa Antolli

Objetivos

- Compreender a importância da ciência de dados e sua aplicação em diferentes setores.
- Aprender a configurar o ambiente de desenvolvimento para programação em Python.
- Familiarizar-se com a sintaxe básica de Python e estruturas de controle.
- Explorar bibliotecas fundamentais para a ciência de dados: NumPy, Pandas, Matplotlib, Seaborn e Scikit-learn.

1. Para iniciarmos a conversa

A ciência de dados é uma área interdisciplinar que combina estatísticas, matemática, programação e conhecimento de domínio para extrair percepções e tomar decisões informadas a partir de grandes volumes de dados. Nos últimos anos, a quantidade de dados gerados cresceu exponencialmente, impulsionada pelo aumento do uso da internet, dispositivos móveis, sensores de IoT e outras tecnologias digitais. Este vasto mar de dados contém padrões valiosos e informações que podem transformar setores como saúde, finanças, marketing, manufatura e muitos outros, tornando a ciência de dados uma habilidade essencial no mercado atual.

Python emergiu como a linguagem de programação preferida para a ciência de dados devido à sua simplicidade, flexibilidade e vasta gama de bibliotecas específicas. Ferramentas como NumPy, Pandas, Matplotlib, Seaborn e Scikit-learn fornecem funcionalidades robustas para manipulação, visualização e modelagem de dados. Além disso, frameworks como TensorFlow e PyTorch permitem a construção de modelos avançados de aprendizado de máquina e inteligência artificial. A sintaxe clara e legível de Python, combinada com sua capacidade de integrar-se facilmente a outras tecnologias, facilita a rápida prototipagem e implementação de soluções de ciência de dados.

2. Introdução ao Python

Neste capítulo, discutiremos as principais abordagens para o desenvolvimento de aplicativos móveis: desenvolvimento web, nativo e híbrido. Cada abordagem tem suas próprias vantagens e desafios, e a escolha entre elas depende das necessidades específicas do projeto.

2.1 Instalação e Configuração do Ambiente de Desenvolvimento

Para começar a programar em Python, é necessário configurar o ambiente de desenvolvimento. A instalação pode variar dependendo do sistema operacional (Windows, macOS, Linux), mas o processo geral é bastante simples. (BANIN, 2018)

2.1.1 Windows

Acesse o site oficial do Python <https://www.python.org/downloads/> e baixe o instalador apropriado para Windows.

- Execute o instalador e certifique-se de marcar a opção “Add Python to PATH”.
- Siga as instruções na tela para completar a instalação.

2.1.2 macOS

- Python 2.x vem pré-instalado no macOS. No entanto, é recomendável instalar a versão mais recente do Python 3.x.
- Use o Homebrew para instalar Python: Abra o terminal e execute “brew install python”.

2.1.3 Linux

- A maioria das distribuições Linux vem com Python pré-instalado. Para instalar ou atualizar para a versão mais recente, use o gerenciador de pacotes da sua distribuição:
- Ubuntu: “sudo apt-get install python3”

- Fedora: “sudo dnf install python3”

2.2 Configuração do Ambiente

Após a instalação do Python, é recomendável configurar um ambiente de desenvolvimento integrado (IDE) para facilitar a escrita e execução do código.

2.2.1 IDEs Populares:

PyCharm: Um dos IDEs mais completos para Python, com recursos avançados de depuração, autocompletar e integração com várias ferramentas.

VS Code (Visual Studio Code): Um editor de código leve e altamente customizável, com suporte para Python através da extensão Python.

2.2.2 Configuração do VS Code:

1. Baixe e instale o Visual Studio Code (<https://code.visualstudio.com/>).
2. Instale a extensão Python para o VS Code.
3. Configure o interpretador Python no VS Code: abra a paleta de comandos (“Ctrl+Shift+P”), digite “Python: Select Interpreter” e selecione o interpretador Python instalado

2.2.3. Ambientes Virtuais:

É uma boa prática usar ambientes virtuais para gerenciar as dependências dos projetos Python. Isso evita conflitos entre pacotes e facilita a manutenção do código.

1. Crie um ambiente virtual com “venv”: “python -m venv myenv”
2. Ative o ambiente virtual:
3. Windows: “myenv\Scripts\activate”
4. macOS/Linux: “source myenv/bin/activate”

3. Sintaxe, Comandos e Estruturas de Decisão e Repetição

Python conta com uma sintaxe clara e intuitiva que facilita a leitura e escrita do código. Vamos explorar os fundamentos da sintaxe Python, incluindo comandos básicos e estruturas de decisão e repetição.

3.1 Sintaxe

Comentários:

Usados para explicar o código, são precedidos pelo símbolo “#”.

```
# Este é um comentário
```

Variáveis e Tipos de Dados:

Python é uma linguagem dinamicamente tipada, o que significa que não é necessário declarar o tipo da variável explicitamente.

```
x = 10 # Inteiro
```

```
y = 3.14 # Float
```

```
name = “Alice” # String
```

```
valido = True # Booleano
```

Operadores:

Python suporta operadores aritméticos ("+", "-", "*", "/"), de comparação ("==", "!=", ">", "<") e lógicos ("and", "or", "not").

Estruturas de Decisão

As estruturas condicionais são usadas para executar diferentes blocos de código com base em certas condições.

"if", "elif", "else" são usados para definir condições e executar blocos de código correspondentes.

```
if x > 0:
```

```
    print("x é positivo")
```

```
elif x == 0:
```

```
    print("x é zero")
```

```
else:
```

```
    print("x é negativo")
```

Estruturas de Repetição – Laço for:

Usado para iterar sobre uma sequência (como uma lista, tupla ou string).

```
for i in range(5):
```

```
    print(i)
```

Estruturas de Repetição – Laço while:

Continua executando um bloco de código enquanto a condição é verdadeira.

```
count = 0  
  
while count < 5:  
  
    print(count)  
  
    count += 1
```

Comandos de Controle de Laço:

“break” para sair do laço.

“continue” para pular para a próxima iteração do laço.

```
for i in range(10):  
  
    if i == 5:  
  
        break  
  
    print(i)
```

4. Listas e Dicionários

Python fornece estruturas de dados integradas, como listas e dicionários, que são de grande relevância para armazenar e manipular coleções de dados.

4.1 Listas

Definição e acesso:

Listas são coleções ordenadas e mutáveis de itens.

```
minha_lista = ["item 1", "item 2", "item 3"]
```

```
print(minha_lista[0]) # Saída: item 1
```

Métodos Comuns:

```
"append(item)", "remove(item)", "pop(index)", "sort()", "reverse()".
```

4.2 Dicionários

Definição e acesso:

Dicionários são coleções desordenadas de pares chave-valor.

```
peessoas = {"nome": "João", "idade": 8, "cidade": "Valinhos"}
```

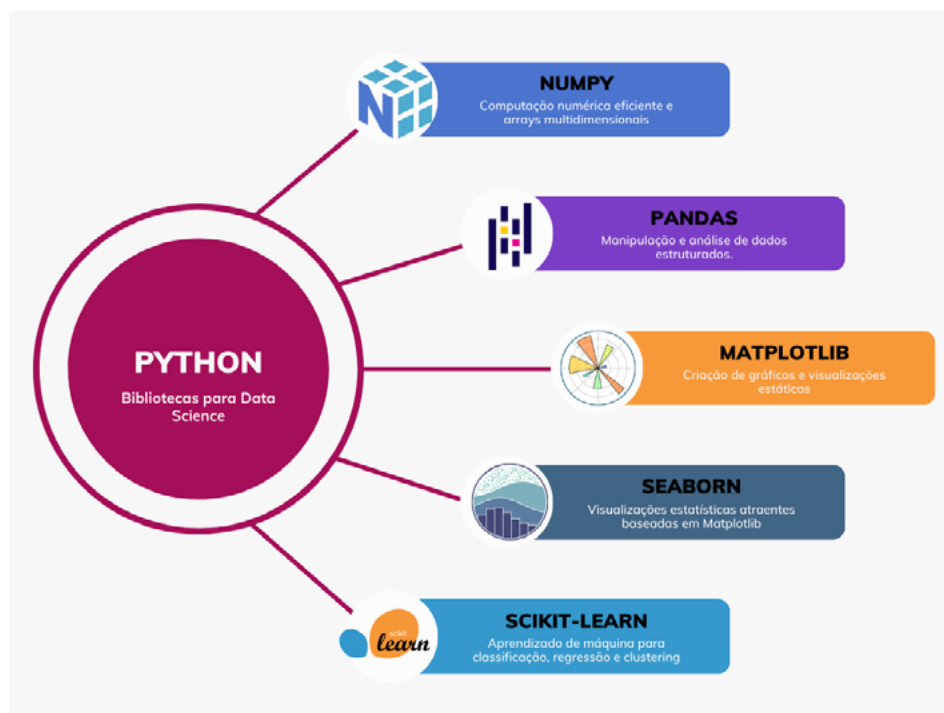
```
print(peessoas["nome"]) # Saída: João
```

Métodos comuns:

```
"keys()", "values()", "items()", "get(key)", "update(dict)".
```

5. Bibliotecas para Ciência de Dados: NumPy, Pandas, Matplotlib, Seaborn

Python é amplamente utilizado na ciência de dados devido à sua rica biblioteca de pacotes e ferramentas que facilitam a análise de dados, visualização e aprendizado de máquina.



Fonte: adaptado de https://numpy.org/images/content_images/ds-landscape.png. Acesso em: 21 ago. 2024.

5.1 NumPy

NumPy é uma biblioteca fundamental para computação científica em Python, amplamente utilizada por cientistas de dados e engenheiros devido à sua capacidade de realizar operações matemáticas e lógicas em grandes coleções de dados de maneira eficiente. Ela introduz uma poderosa classe de array multidimensional chamado `ndarray`, que permite a manipulação de grandes datasets com alto desempenho. Além de operações básicas de array, NumPy oferece uma vasta gama de funções matemáticas, como álgebra linear, transformadas de Fourier e geração de números aleatórios. A biblioteca é projetada para interoperabilidade com outras ferramentas e bibliotecas, como Pandas,

Matplotlib e Scikit-learn, formando a espinha dorsal do ecossistema de ciência de dados em Python. A eficiência de NumPy em processamento de grandes volumes de dados e sua sintaxe intuitiva a tornam indispensável para tarefas de análise e modelagem de dados complexos (NumPy, 2024).

5.2 Pandas

Pandas é uma biblioteca para manipulação e análise de dados em Python, especialmente popular entre cientistas de dados e analistas. Ela fornece estruturas de dados de alto nível, como Series e DataFrame, que facilitam o armazenamento e a manipulação de dados tabulares de maneira eficiente e intuitiva. Com Pandas, é possível realizar operações complexas de limpeza, filtragem, agregação e transformação de dados com poucas linhas de código, além de oferecer funcionalidades robustas para leitura e escrita de dados em diversos formatos, como CSV, Excel e SQL (Behrman, 2023).

5.3 Matplotlib

Segundo Behrman (2023), Matplotlib é uma biblioteca amplamente utilizada para criação de visualizações estáticas, animadas e interativas em Python. Ela fornece uma interface versátil que permite a geração de gráficos de alta qualidade, incluindo gráficos de linha, barras, dispersão, histogramas e muito mais. Matplotlib é altamente configurável, permitindo aos usuários personalizar praticamente todos os aspectos de suas visualizações, desde cores e estilos de linha até rótulos e legendas. Esta flexibilidade é especialmente útil para cientistas de dados e analistas que precisam comunicar insights de maneira clara e visualmente atraente.

5.4 Seaborn



Seaborn é uma biblioteca poderosa e intuitiva para a criação de visualizações estatísticas em Python, construída sobre Matplotlib. Ela simplifica a criação de gráficos atraentes e informativos, oferecendo uma interface de alto nível para desenhar visualizações estatísticas complexas com apenas algumas linhas de código. Seaborn facilita a visualização de distribuições de dados, relações entre variáveis e comparações de categorias, suportando gráficos como mapas de calor, gráficos de violino, gráficos de dispersão com regressão e gráficos de barras categóricas (Seaborn, 2024)

5.5 Scikit-learn

De acordo com o site Scikit-learn (2024), Scikit-learn é uma das bibliotecas mais populares e amplamente utilizadas em Python para aprendizado de máquina e análise de dados. Ela fornece uma coleção abrangente de algoritmos eficientes e ferramentas para tarefas como classificação, regressão, *clustering*, redução de dimensionalidade, seleção de modelo e pré-processamento de dados. A simplicidade da interface de Scikit-learn, combinada com sua robustez, permite que tanto iniciantes quanto especialistas implementem rapidamente modelos de aprendizado de máquina, realizem experimentos e avaliem o desempenho de diferentes técnicas. A biblioteca é construída sobre NumPy, SciPy e Matplotlib, garantindo integração perfeita com outras ferramentas do ecossistema Python, e é constantemente atualizada pela comunidade de código aberto, mantendo-se na vanguarda das inovações em aprendizado de máquina.

Conectando à Realidade:

A empresa Tech API Solutions é uma consultoria especializada em ciência de dados, que ajuda empresas de diversos setores a extrair insights

valiosos de seus dados. Com a crescente demanda por habilidades em ciência de dados, a empresa decidiu oferecer um treinamento básico em programação Python, focado em manipulação de dados e visualização. O objetivo do treinamento é capacitar analistas de dados e profissionais de TI para que possam utilizar Python e suas bibliotecas principais na análise e interpretação de grandes volumes de dados.

Caso:

A Tech API Solutions foi contratada por uma empresa do setor financeiro para treinar uma equipe para realizar uma análise de seus dados de transações. A empresa deseja identificar padrões de comportamento dos clientes, detectar fraudes e melhorar a eficiência de suas operações. O projeto foi dividido em quatro partes principais:

Configuração do ambiente de desenvolvimento:

Instalação do Python e configuração de um ambiente de desenvolvimento integrado (IDE).

Criação de ambientes virtuais para gerenciar dependências do projeto.

Introdução à sintaxe de Python:

Comandos básicos, variáveis e tipos de dados.

Estruturas de decisão e repetição.

Manipulação de dados com bibliotecas Python:

Uso do NumPy e Pandas para manipulação e análise de dados.

Visualização de dados com Matplotlib e Seaborn.

Aplicação de modelos de aprendizado de máquina:

Utilização do Scikit-learn para construção e avaliação de modelos preditivos.

- Análise:

A equipe de desenvolvimento analisou os seguintes pontos:

- Escolha da Ferramenta:

Python foi escolhido pela sua simplicidade, flexibilidade e vasta gama de bibliotecas específicas para ciência de dados.

- Requisitos do Projeto:

Configuração de um ambiente de desenvolvimento consistente.

Necessidade de manipular grandes volumes de dados e gerar visualizações claras e informativas.

Implementação de modelos de aprendizado de máquina para prever comportamentos e detectar fraudes.

- Desafios Técnicos:

Instalação e configuração correta das ferramentas e bibliotecas.

Garantir que os analistas de dados compreendam a sintaxe e os conceitos básicos de Python.

Integração eficiente das bibliotecas para manipulação e análise de dados.

- Planejamento das Aulas:

Estruturação do conteúdo do treinamento em módulos claros e progressivos.

Criação de exemplos práticos e exercícios para reforçar o aprendizado.
Solução:

Desenvolvimento do Treinamento:

Configuração do Ambiente de Desenvolvimento:

Instalação do Python e configuração do VS Code como IDE principal, bem como as extensões para Python.

Criação de ambientes virtuais utilizando venv para gerenciar dependências.

Utilização do Jupyter Notebook para facilitar a visualização e execução do código.

- Sintaxe de Python:

Explicação detalhada de variáveis, tipos de dados, operadores, estruturas de decisão (if, elif, else) e laços de repetição (for, while).

Exemplos práticos para ilustrar cada conceito.

- Manipulação de Dados com Bibliotecas Python:

Introdução ao NumPy para operações matemáticas e manipulação de arrays.

Uso do Pandas para manipulação de dados tabulares, leitura e escrita de arquivos CSV e Excel.

Criação de visualizações com Matplotlib e Seaborn para explorar distribuições de dados e relações entre variáveis.

- Aplicação de Modelos de Aprendizado de Máquina:

Introdução ao Scikit-learn e sua interface simples para implementar algoritmos de aprendizado de máquina.

Exemplos práticos de classificação, regressão e clustering.

Avaliação de modelos e ajuste de parâmetros.



Referências:

BANIN, Sérgio L. **Python 3 – Conceitos e Aplicações – Uma abordagem didática**. SRV Editora LTDA, 2018. E-book. ISBN 9788536530253. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9788536530253/>. Acesso em: 20 maio 2024.

BEHRMAN, Kennedy R. **Fundamentos de Python para ciência de dados**. Grupo A, 2023. E-book. ISBN 9788582605974. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9788582605974/>. Acesso em: 20 maio 2024.

NUMPY. **NumPy**. Disponível em: <https://numpy.org/>. Acesso em: 23 jul. 2024.

SEABORN. **Seaborn**: Statistical Data Visualization. Disponível em: <https://seaborn.pydata.org/>. Acesso em: 23 jul. 2024.

SCIKIT-LEARN. **Scikit-learn**: Machine Learning in Python. Disponível em: <https://scikit-learn.org/>. Acesso em: 23 jul. 2024.

Big Data e Apache Spark

Autoria: Rodrigo Amaral Rocha

Leitura crítica: Elisa Antolli

Objetivos

- Explorar o conceito de Big Data, incluindo os desafios associados ao processamento de grandes volumes de dados em tempo real. Discutir os “5 Vs” do Big Data (volume, velocidade, variedade, veracidade e valor) e como eles influenciam a análise de dados em larga escala.
- Entender o Apache Spark como uma ferramenta poderosa para processamento de Big Data. Discutir as principais características do Spark, como sua capacidade de processamento distribuído, tolerância a falhas e suporte a múltiplos tipos de dados.
- Saber como o Apache Spark é utilizado para resolver problemas de Big Data, como análise de dados em tempo real, processamento de grandes conjuntos de dados e machine learning.
- Configurar ambientes de desenvolvimento, escrita e execução de scripts em Spark usando linguagens como Python ou Scala, e a realização de análises básicas em conjuntos de dados de exemplo.

1. Para iniciarmos a conversa

Nos últimos tempos, presenciamos uma mudança significativa, porém discreta, que está redefinindo nossa compreensão e uso dos dados. Essa transformação é impulsionada pelo Big Data, um conceito que vai muito além da simples quantidade de informações disponíveis. O Big Data refere-se a conjuntos de dados enormes e complexos que desafiam as abordagens tradicionais de processamento e análise de dados.

Qual é a importância do Big Data nos dias de hoje? A resposta está na capacidade sem precedentes de obter percepções valiosas desses vastos e diversos conjuntos de dados. Em um mundo cada vez mais digitalizado, empresas, governos e instituições estão coletando dados em uma escala nunca vista. Desde transações financeiras até interações em redes sociais, passando por informações de sensores em tempo real, a quantidade e variedade de dados gerados diariamente são verdadeiramente impressionantes.

No entanto, simplesmente acumular dados não é o bastante. O verdadeiro potencial do Big Data está na habilidade de transformar esses dados em informações práticas. Ao analisar padrões complexos, correlações sutis e tendências emergentes, as organizações podem tomar decisões mais embasadas e estratégicas. Isso não só melhora a eficiência operacional e reduz os custos, como também impulsiona a inovação, abre novas oportunidades no mercado e oferece uma vantagem competitiva significativa.

Os cinco aspectos principais do Big Data – volume, velocidade, variedade, veracidade e valor – são a base dessa mudança. Eles explicam as características essenciais dos dados de Big Data e destacam os desafios e benefícios ligados à sua utilização eficaz.

- **Volume:** a quantidade massiva de dados gerados diariamente. Exemplos incluem dados de redes sociais, transações financeiras e logs de sensores IoT.
- **Velocidade:** a velocidade com que novos dados são gerados e precisam ser processados. Isso é crítico em aplicações como transações financeiras em tempo real e monitoramento de segurança.
- **Variedade:** a diversidade de tipos de dados. Dados podem ser estruturados (bancos de dados), semiestruturados (XML, JSON) ou não estruturados (texto, imagens e vídeos).
- **Veracidade:** a confiabilidade dos dados. É importante assegurar a precisão e a qualidade dos dados para análises significativas.
- **Valor:** A capacidade de extrair insights úteis e gerar valor a partir dos dados.

2. Apache Spark

Para lidar com os desafios do Big Data, ferramentas e frameworks avançados são necessários. O Apache Spark é uma dessas ferramentas, destacando-se por sua capacidade de processar grandes volumes de dados de forma eficiente e rápida.

Apache Spark é um framework de código aberto para processamento distribuído de dados em larga escala. Ele foi projetado para ser rápido e de uso geral, permitindo tanto processamento em *batch* quanto em tempo real. A arquitetura do Spark é composta por várias camadas que interagem para fornecer um ambiente robusto de processamento de dados. Veremos como ela é composta a seguir.

2.1 Arquitetura

- Driver:

O Driver é a aplicação principal que cria o contexto Spark, define as operações no RDDs e DataFrames, e solicita a execução dessas operações. Ele é responsável pela tradução do código em tarefas executáveis.

- Cluster Manager:

O Cluster Manager gerencia os recursos do cluster e aloca recursos para as aplicações Spark. Spark pode ser executado em diferentes gerenciadores de cluster, como Hadoop YARN, Apache Mesos e o próprio Standalone Cluster Manager do Spark.

- Executors:

Os Executors são processos que executam as tarefas atribuídas pelo Driver. Eles armazenam dados em cache e realizam operações de processamento de dados.

- Tasks:

As Tasks são as unidades de trabalho que o Executor executa. Cada operação em um RDD ou DataFrame é decomposta em várias tarefas que são distribuídas pelos nós do cluster.

2.2 Componentes

O Apache Spark é composto por vários componentes principais que fornecem diferentes funcionalidades:

- Spark Core:

O núcleo do Spark fornece funcionalidades básicas de processamento de dados, como operações em RDDs, gerenciamento de memória, e execução de tarefas. É a base sobre a qual todos os outros componentes são construídos.

- Spark SQL:

Um módulo para processamento estruturado de dados. Ele permite consultas SQL em DataFrames e RDDs, integração com bancos de dados JDBC, e suporte para formatos de dados como JSON e Parquet.

- MLlib:

Uma biblioteca de aprendizado de máquina escalável que fornece algoritmos comuns como classificação, regressão, clustering e filtragem colaborativa. Ela também inclui utilitários para avaliação de modelos e pipelines de ML.

- Spark Streaming:

Um módulo para processamento de dados em tempo real. Ele permite a construção de pipelines de processamento de dados que podem ingerir dados de fontes como Kafka, Flume, e sockets TCP.

- GraphX:

Uma API para processamento de grafos que permite a execução de algoritmos de grafos em grande escala, como PageRank, clustering e busca de caminhos mais curtos.

3. RDDs (*Resilient Distributed Datasets*)

Os RDDs são a principal abstração de dados do Spark. Eles representam uma coleção distribuída e imutável de objetos que

podem ser processados em paralelo. Algumas características importantes dos RDDs são:

- **Imutabilidade:**

Uma vez criado, um RDD não pode ser alterado. Qualquer transformação em um RDD gera um novo RDD.

- **Particionamento:**

Os RDDs são automaticamente particionados pelo Spark, permitindo que as operações sejam distribuídas e processadas em paralelo nos nós do cluster.

- **Linhas de Proveniência:**

Os RDDs mantêm a linhagem das transformações aplicadas a eles, o que permite a recomputação de partes dos dados em caso de falhas.

- **Persistência:**

Os RDDs podem ser armazenados em memória ou disco, permitindo que os dados sejam reutilizados em várias operações.

Exemplo de criação e uso de RDDs em Python:

```
from pyspark import SparkContext

sc = SparkContext("local", "RDD Example")

# Criação de um RDD a partir de uma coleção

data = [1, 2, 3, 4, 5]

rdd = sc.parallelize(data)
```

```
# Aplicação de uma transformação

squared_rdd = rdd.map(lambda x: x 2)

# Ação para coletar os resultados

result = squared_rdd.collect()

print(result)
```

4. DataFrames e Spark SQL

Os DataFrames são uma abstração de dados de alto nível no Spark, similar aos DataFrames do Pandas em Python ou às tabelas em bancos de dados relacionais. Eles são mais expressivos e eficientes do que os RDDs para processamento estruturado de dados. Spark SQL permite realizar consultas SQL diretamente sobre DataFrames, facilitando a integração com aplicações que já utilizam SQL.

4.1 Criação da DataFrames

- **A partir de uma coleção:**

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("Exemplo de DataFrame")\
    .getOrCreate()

data = [("Rodrigo", 34), ("Vivian", 45), ("Catarina", 29)]

columns = ["Nome", "Idade"]
```

```
df = spark.createDataFrame(data, columns)
```

```
df.show()
```

- **A partir de um arquivo CSV:**

```
df = spark.read.csv("dados.csv", header=True, inferSchema=True)
```

```
df.show()
```

- **Consultas com Spark SQL**

```
df.createOrReplaceTempView("people")
```

- # Consultando usando SQL**

```
result = spark.sql("SELECT * FROM pessoas WHERE idade > 30")
```

```
result.show()
```

5. Operações e Transformações

No Spark, as operações em RDDs e DataFrames são divididas em dois tipos: transformações e ações.

5.1 Transformações

Transformações criam um novo RDD a partir de um RDD existente. Elas são "lazy", o que significa que não são executadas até que uma ação seja chamada. Exemplos incluem:

`map()`:

- Aplica uma função a cada elemento do RDD.

```
rdd2 = rdd.map(lambda x: x * 2)
```

filter():

- Filtra elementos do RDD com base em uma condição.

```
rdd3 = rdd.filter(lambda x: x > 2)
```

flatMap():

- Similar ao "map()", mas pode retornar múltiplos itens para cada entrada.

```
rdd4 = rdd.flatMap(lambda x: [x, x2])
```

5.2 Ações

- Ações computam um resultado baseado em um RDD e retornam um valor para o Driver. Exemplos incluem:

- collect():

- Retorna todos os elementos do RDD.

```
result = rdd.collect()
```

- count():

- Retorna o número de elementos no RDD.

```
num_elements = rdd.count()
```

first():

- Retorna o primeiro elemento do RDD.

```
first_element = rdd.first()
```

Conectando à Realidade:

Nos últimos anos, uma empresa de mídia social fictícia, denominada “Jund Connect”, tem experimentado um crescimento exponencial em sua base de usuários e, consequentemente, no volume de dados gerados diariamente. Esses dados incluem registros de interações dos usuários, como visualizações de postagens, curtidas, comentários e compartilhamentos. Com o objetivo de melhorar a personalização dos feeds de notícias, entender o comportamento dos usuários de forma mais profunda e otimizar a entrega de anúncios, a empresa enfrenta o desafio de analisar esses dados em larga escala e em tempo real.

Caso:

Para lidar com esses desafios, a equipe de análise de dados da Jund Connect opta por usar o Apache Spark, uma plataforma de Big Data amplamente conhecida por sua capacidade de processar grandes volumes de dados eficaz e escalonável. Eles iniciam o processo coletando e preparando um conjunto de dados fictícios que representam as atividades dos usuários na plataforma.

Com o Spark Streaming, uma funcionalidade do Apache Spark voltada para processamento em tempo real, a equipe estabelece uma sequência de processamento que permite analisar as interações dos usuários conforme elas ocorrem. Isso envolve contar as interações por usuário em intervalos específicos, identificar tendências emergentes, detectar eventos significativos (como picos de atividade) e monitorar constantemente a atividade na plataforma.

Ao mesmo tempo, utilizando o Spark SQL, a equipe realiza análises em lotes nos dados históricos para investigar padrões comportamentais ao longo do tempo. Eles fazem consultas elaboradas para determinar a popularidade das postagens, segmentar os usuários com base em interesses compartilhados e fazer previsões sobre tendências futuras com base nos dados coletados.

Solução:

A solução implementada usando Apache Spark permite à Jund Connect:

Escalabilidade: lidar eficientemente com o aumento no volume de dados à medida que a base de usuários continua a crescer, escalando horizontalmente conforme necessário.

Flexibilidade: adaptar-se facilmente a diferentes tipos de análises, combinando processamento em tempo real com análises de batch para insights abrangentes.

Entrega de valor: melhorar a personalização do feed de notícias para os usuários, entender e antecipar seus interesses, além de otimizar a entrega de anúncios de forma mais eficaz e relevante.

Em suma, ao adotar Apache Spark, a Jund Connect não apenas enfrenta seus desafios de Big Data de maneira eficaz, mas também se posiciona para aproveitar ao máximo os dados gerados por seus usuários, promovendo uma experiência mais personalizada e aprimorada na plataforma de mídia social.

Referências:

APACHE SPARK. **Apache Spark – Unified Engine for large-scale data analytics.** Disponível em: <https://spark.apache.org/>. Acesso em: 1 jul. 2024.

BEHRMAN, Kennedy R. **Fundamentos de Python para ciência de dados.**

Grupo A, 2023. E-book. ISBN 9788582605974. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9788582605974/>. Acesso em: 20 maio 2024.

PEREIRA, Mariana A. *et al.* **Framework de Big Data.** Porto Alegre: Grupo

A, 2020. E-book. ISBN 9786556900803. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9786556900803/>. Acesso em: 1 jul. 2024.

PySpark – Análise de Dados

Autoria: Rodrigo Amaral Rocha

Leitura crítica: Elisa Antolli

Objetivos

- Compreender o uso de DataFrames no Apache Spark para manipulação eficiente e estruturada de grandes volumes de dados.
- Aprender a ler e escrever diferentes tipos e formatos de dados, como JSON, CSV, Parquet e Avro, usando Apache Spark.
- Entender como realizar operações de filtragem, agregação e união em DataFrames para análises de dados complexas e eficientes.
- Conhecer como o Apache Spark realiza processamento distribuído para manipular e analisar grandes conjuntos de dados de maneira eficiente.
- Explorar a biblioteca MLlib do Apache Spark para aplicar algoritmos de aprendizado de máquina em grandes volumes de dados.
- Implementar pipelines de dados no Apache Spark para automatizar e otimizar fluxos de trabalho de análise de dados.

1. Introdução

PySpark é a ferramenta que conecta Python ao Apache Spark, proporcionando uma maneira poderosa de analisar dados e realizar *machine learning* em grande escala. Originado no AMPLab da Universidade da Califórnia, em Berkeley, e mantido pela Apache Software Foundation, o Spark se destaca por sua velocidade e capacidade de processamento em memória. Por meio do PySpark, desenvolvedores e cientistas de dados podem aproveitar toda a funcionalidade robusta do Spark usando a linguagem Python, incluindo manipulação de dados com DataFrames e Datasets, execução de consultas SQL e aplicação de algoritmos de *machine learning* com MLlib.

Com o PySpark, é possível processar e analisar grandes conjuntos de dados de forma eficaz. A interface simplificada do PySpark combina a facilidade do Python com a escalabilidade e desempenho do Spark. Esse recurso é especialmente útil em situações que demandam processamento contínuo em tempo real, análises sofisticadas e *machine learning* em larga escala. Além disso, o PySpark oferece suporte para operações distribuídas, possibilitando o processamento dos dados em um cluster de computadores para análise eficiente de grandes conjuntos que não caberiam na memória individual de um computador.

2. DataFrames

Segundo o site Apache Spark (2024), os DataFrames em PySpark representam uma estrutura de dados distribuída que viabiliza a manipulação eficaz e organizada de extensos conjuntos de dados. Modelados a partir dos DataFrames do R e do Pandas Python, os DataFrames no PySpark são imutáveis e distribuídos, tornando-os ideais para operações em Big Data em grande escala. As principais características dos DataFrames são:

1. Estruturados e com esquema definido:

Os DataFrames organizam os dados em colunas nomeadas, assemelhando-se a uma tabela em um banco de dados relacional. Cada DataFrame conta com um esquema que especifica os nomes e tipos de dados de cada coluna.

2. Distribuídos:

Os DataFrames são distribuídos entre diversos nós de um cluster, possibilitando a manipulação e o processamento eficiente de grandes volumes de dados que excedem a capacidade da memória de uma única máquina.

3. Imutáveis:

Uma vez criado, um DataFrame não pode ser modificado. Todas as operações realizadas em um DataFrame resultam na criação de um novo DataFrame.

4. Operações avançadas:

Oferecem suporte a uma variedade de operações avançadas como seleção, filtragem, agregação e junção dos dados, simplificando o manuseio de conjuntos complexos com eficiência.

Vantagem dos DataFrames:

Desempenho otimizado: empregam o Otimizador Catalyst para automaticamente otimizar as consultas, garantindo desempenho superior nas operações mais complexas.

2.2 Criação de DataFrames



A criação de DataFrames é uma tarefa fundamental ao trabalhar com PySpark. Eles permitem que os dados sejam manipulados e analisados de maneira eficiente e escalável, aproveitando a capacidade de processamento paralelo do Apache Spark.

A criação de DataFrames é essencial para aproveitar a poderosa funcionalidade de processamento de dados do PySpark. Ao entender como criar e configurar DataFrames, você estará preparado para realizar operações complexas de análise de dados e *machine learning* em grandes volumes de dados.

Exemplos de criação de DataFrames:

A partir de uma lista de dicionários:

```
from pyspark.sql import SparkSession

# Inicializar uma SparkSession

spark = SparkSession.builder.
appName("ExemploDataFrame").getOrCreate()

# Criar uma lista de tuplas

dados = [

    ("João", 25),

    ("Maria", 30),

    ("Ale", 35)

]

# Definir o esquema (nome das colunas)

colunas = ["Nome", "Idade"]

# Criar o DataFrame

df = spark.createDataFrame(dados, colunas)

# Mostrar o conteúdo do DataFrame

df.show()
```

A partir de um arquivo CSV:

```
from pyspark.sql import SparkSession

# Inicializar uma SparkSession
```

```
spark = SparkSession.builder.appName("ExemploDataFrameCSV").  
getOrCreate()  
  
# Caminho para o arquivo CSV  
  
caminho_csv = "caminho/para/seu/arquivo.csv"  
  
# Ler o arquivo CSV em um DataFrame  
  
df = spark.read.csv(caminho_csv, header=True, inferSchema=True)  
  
# Mostrar o conteúdo do DataFrame  
  
df.show()
```

2.1 Leitura e Escrita de Diferentes Tipos e Formatos de Dados

PySpark suporta a leitura e escrita de vários formatos de dados, incluindo CSV, JSON, Parquet, Avro e muito mais. Isso facilita a integração com diferentes fontes de dados e o armazenamento de resultados em diversos formatos.

2.2.1 Tipos de Formatos de Dados Suportados por PySpark

PySpark suporta a leitura e escrita de vários formatos de dados, facilitando a integração com diferentes fontes de dados e o armazenamento de resultados em diversos formatos. A seguir, você tem alguns dos principais formatos suportados:

CSV (Comma-Separated Values):

CSV é um formato amplamente utilizado para armazenar dados tabulares em texto simples, em que cada linha representa um registro e as colunas são separadas por vírgulas.

Vantagem: simples e fácil de ler e escrever.

Desvantagem: não conta com suporte nativo para tipos de dados complexos ou hierárquicos.

JSON (JavaScript Object Notation):

JSON é um formato leve de intercâmbio de dados que é fácil para humanos lerem e escreverem, e fácil para máquinas interpretarem e gerarem.

Vantagem: suporta estruturas de dados complexas e aninhadas.

Desvantagem: pode ser menos eficiente em termos de espaço comparado a outros formatos binários.

Parquet:

Parquet é um formato de armazenamento colunar otimizado para leitura e escrita rápida de grandes volumes de dados.

Vantagem: alto desempenho para consultas analíticas, suporta compressão eficiente.

Desvantagem: pode ser mais complexo de escrever e configurar comparado a formatos como CSV.

Avro:

Avro é um formato de serialização de dados que suporta esquemas dinâmicos, permitindo a evolução dos dados.

Vantagem: bom suporte para compressão e integração com sistemas de Big Data.

Desvantagem: requer a definição de esquemas, o que pode adicionar complexidade.

Outros Formatos:

PySpark também suporta outros formatos de dados, como ORC, XML, e bancos de dados relacionais via JDBC.

Vantagem: flexibilidade para trabalhar com diferentes sistemas de armazenamento de dados.

Desvantagem: cada formato pode ter suas próprias nuances e requisitos de configuração.

Esses formatos oferecem flexibilidade e eficiência ao manipular e armazenar grandes volumes de dados. Escolher o formato correto depende do caso de uso específico, do desempenho necessário e da complexidade dos dados a serem processados.

Exemplo 1: Leitura de Dados

CSV

```
df = spark.read.csv("arquivo.csv", header=True, inferSchema=True)`
```

JSON

```
df = spark.read.json("arquivo.json")
```

Parquet:

```
df = spark.read.parquet("arquivo.parquet")
```

Exemplo 2: Escrita de Dados

CSV:

```
df.write.csv("arquivo.csv", header=True)
```

JSON:

```
df.write.json("arquivo.json")
```

Parquet:

```
df.write.parquet("arquivo.parquet")
```

2.3 Filtragem, Agregação e Operações de União

Manipular e transformar dados são tarefas essenciais em análise de dados. PySpark oferece uma ampla gama de operações para filtrar, agregar e unir DataFrames.

2.3.1 Filtragem

Filtragem em PySpark é o processo de selecionar linhas específicas de um DataFrame ou RDD (*Resilient Distributed Dataset*) com base em condições definidas. Esta operação permite extrair subconjuntos de dados que atendem a critérios específicos, facilitando a análise de dados relevantes e a remoção de dados indesejados ou irrelevantes.

A filtragem é realizada utilizando o método `filter()`, que aplica uma função ou condição lógica a cada linha do DataFrame ou RDD. Apenas as linhas que atendem à condição especificada são incluídas no resultado final.

Exemplo: Filtro simples:

```
df_filtered = df.filter(df.age > 30)

df_filtered.show()
```

Exemplo: Filtro com múltiplas condições:

```
df_filtered = df.filter((df.idade > 30) & (df.nome == "Joao"))

df_filtered.show()
```

2.3.2 Agregação

A agregação no PySpark se refere ao processo de realizar operações estatísticas ou matemáticas em um conjunto de dados para resumir ou combinar informações. Isso envolve a execução de operações como soma, média, contagem, mínimo e máximo em grupos de dados para gerar um valor representativo único ou um resumo. As operações de agregação são comumente empregadas para analisar grandes conjuntos de dados de forma eficaz, permitindo insights rápidos e informados. Por exemplo, ao agrupar vendas por região e calcular a média das vendas em cada grupo, é viável identificar tendências e padrões de desempenho regional. No PySpark, essas operações são executadas por meio de métodos como **groupBy()**, **agg()** e **count()**.

Exemplo: Agregação simples

```
df_agg = df.groupBy("nome").agg({"idade": "mean"})

df_agg.show()
```

Resultado:

```
+-----+-----+
```

```
| nome | avg(idade) |
```

```
+-----+-----+
```

```
| Maria | 35.5 |
```

```
| João | 28.0 |
```

```
| Ale | 42.0 |
```

```
| Ana | 31.0 |
```

```
| Carlos | 39.5 |
```

```
+-----+-----+
```

Exemplo: Agregação com funções específicas

```
from pyspark.sql.functions import avg, max
```

```
df_agg = df.groupBy("nome").agg(avg("idade"), max("idade"))
```

```
7. df_agg.show()
```

Resultado:

```
+-----+-----+-----+
```

```
| nome | avg(idade) | max(idade) |
```

```
+-----+-----+-----+
```

```
| Maria | 35.5 | 42 |
```

| João | 28.0 | 28 |

| Ana | 31.0 | 31 |

| Carlos | 45.5 | 51 |

| Juliana | 29.0 | 29 |

| Ale | 37.3 | 40 |

+-----+-----+-----+

Operações de União

A operação de união em PySpark é usada para combinar duas ou mais DataFrames ou RDDs com o mesmo esquema, concatenando suas linhas. Esse processo é essencial quando se deseja reunir dados de múltiplas fontes em um único DataFrame ou RDD para análise conjunta. A união preserva todas as linhas dos DataFrames ou RDDs envolvidos, criando um novo DataFrame ou RDD resultante. É particularmente útil para mesclar dados divididos em diferentes arquivos ou para combinar resultados de diferentes partes de um pipeline de processamento de dados. Em PySpark, a operação de união é realizada usando o método **union()**.

Exemplo: União de DataFrames

```
df1 = spark.createDataFrame([("Rodrigo", 25), ("Silvio", 30)],
                             ["nome", "idade"])
```

```
df2 = spark.createDataFrame([("Rosana", 35), ("Patricia", 40)],
                             ["nome", "idade"])
```

```
df_union = df1.union(df2)
```

```
df_union.show()
```

Exemplo: União com remoção de duplicatas

```
df_union = df1.union(df2).dropDuplicates()
```

```
df_union.show()
```

3. Processamento Distribuído

Uma das maiores vantagens do PySpark é sua capacidade de processar grandes volumes de dados de maneira distribuída, utilizando a capacidade de processamento paralelo dos clusters.

O processamento distribuído envolve a capacidade de realizar operações de processamento de dados em paralelo, utilizando um conjunto de computadores interligados. Em vez de lidar com análises e manipulações de dados em um único ponto, o PySpark divide as tarefas em segmentos menores que são distribuídos entre diferentes pontos do conjunto. Cada ponto processa uma parte dos dados de forma autônoma e simultânea, acelerando consideravelmente o tempo de execução e viabilizando o gerenciamento eficiente de grandes quantidades de dados que seriam impraticáveis para processar em um único computador.

Um cluster é formado por vários computadores conectados que colaboram para realizar tarefas computacionais. Cada computador dentro do cluster é conhecido como nó. Existem dois tipos principais de nós em um cluster: mestre e trabalhador. O nó mestre supervisiona a distribuição das tarefas e acompanha a execução das operações, enquanto os nós trabalhadores executam as tarefas designadas a eles (Apache Spark, 2024).

Elementos do processamento distribuído no PySpark:

8. Nó mestre:

Encarregado da coordenação das atividades no conjunto.

Distribui as tarefas para os pontos trabalhadores e monitora seu andamento.

9. Nós trabalhadores:

Realizam as tarefas distribuídas pelo nó mestre.

Processam partes dos dados simultaneamente.

RDD (conjuntos de dados distribuídos resilientes):

São estruturas de dados distribuídas e imutáveis que auxiliam no processamento em paralelo e na tolerância a falhas. Eles dividem os dados em pedaços que são espalhados pelos nós do agrupamento.

10. Agentes (executores):

São as unidades encarregadas de realizar as tarefas designadas a cada nó de processamento. Eles cuidam da memória e dos recursos computacionais necessários para executar as operações.

Exemplo: Configuração básica de um cluster Spark

```
from pyspark import SparkConf, SparkContext
```

```
from pyspark.sql import SparkSession
```

```
conf = SparkConf().setAppName("Meu App").setMaster("spark://  
master:7077")
```

```
sc = SparkContext(conf=conf)
```

```
spark = SparkSession(sc)
```

Exemplo: Leitura e processamento de dados em um cluster

```
df = spark.read.csv("hdfs://caminho/arquivo.csv", header=True,  
inferSchema=True)
```

```
df_filtered = df.filter(df.idade > 30)
```

```
df_filtered.show()
```

4. MLlib: *Machine Learning*

De acordo com Pereira (2020), MLlib é a biblioteca de *machine learning* do Apache Spark, projetada para fornecer ferramentas escaláveis e de alto desempenho para aprendizado de máquina e análise de dados. Ela oferece uma ampla gama de algoritmos e utilitários que facilitam a implementação de tarefas comuns de aprendizado de máquina em grandes volumes de dados. A biblioteca é construída para se integrar perfeitamente com a API do Spark, aproveitando as capacidades de processamento distribuído e em memória do Spark para executar algoritmos de *machine learning* de maneira eficiente.

Algoritmos de *Machine Learning*

1. Regressão Linear

A regressão linear é uma técnica estatística e de aprendizado de máquina utilizada para modelar a relação entre uma variável dependente (alvo) e uma ou mais variáveis independentes (preditores). Em termos mais simples, a regressão linear tenta encontrar a melhor

linha reta (ou hiperplano no caso de múltiplas variáveis) que descreve como a variável alvo varia conforme as variáveis preditoras mudam.

Exemplo:

```
from pyspark.ml.regression import LinearRegression

df = spark.read.csv("dados.csv", header=True, inferSchema=True)

lr = LinearRegression(featuresCol="features", labelCol="label")

model = lr.fit(df)

predictions = model.transform(df)

predictions.show()
```

2. Clusterização K-Means:

Segundo o site Apache Spark (2024), clusterização K-Means é uma técnica de aprendizado de máquina não supervisionada utilizada para particionar um conjunto de dados em K clusters (ou grupos) distintos. O algoritmo funciona iterativamente para atribuir cada ponto de dado a um dos K clusters com base na minimização da variabilidade dentro dos clusters. Inicialmente, K centros de cluster (centroides) são escolhidos aleatoriamente. Em seguida, cada ponto de dado é atribuído ao centro de cluster mais próximo, e os centroides são recalculados como a média dos pontos atribuídos a cada cluster. Este processo de atribuição e atualização dos centroides continua até que as posições dos centroides se estabilizem ou atinjam um número máximo de iterações. O K-Means é amplamente utilizado em diversas áreas, como segmentação de mercado, análise de padrões e compressão de imagens, devido à sua simplicidade e eficiência. Exemplo:

```
from pyspark.ml.clustering import KMeans

df = spark.read.csv("dados.csv", header=True, inferSchema=True)

kmeans = KMeans().setK(3).setSeed(1)

model = kmeans.fit(df)

predictions = model.transform(df)

predictions.show()
```

4. Pipelines

Os Pipelines em PySpark compõem uma ferramenta para criar fluxos de trabalho estruturados de *machine learning*. Eles permitem combinar várias etapas de processamento de dados e aprendizado de máquina em um fluxo contínuo, desde a preparação dos dados até a avaliação do modelo. A principal vantagem de usar pipelines é a capacidade de gerenciar e automatizar todo o processo de machine learning de maneira eficiente e reproduzível (Apache Spark, 2024).

Exemplo de Pipeline

```
# Importações necessárias

from pyspark.sql import SparkSession

from pyspark.ml import Pipeline

from pyspark.ml.feature import VectorAssembler, StandardScaler

from pyspark.ml.regression import LinearRegression
```

```
from pyspark.ml.evaluation import RegressionEvaluator

# Inicializar uma SparkSession

spark = SparkSession.builder.appName("ExemploPipeline").
getOrCreate()

# Carregar os dados de exemplo

# Substitua "caminho/para/seu/arquivo.csv" pelo caminho real do
seu arquivo CSV

dados = spark.read.csv("caminho/para/seu/arquivo.csv",
header=True, inferSchema=True)

# Selecionar colunas relevantes

# Substitua "coluna1", "coluna2", "coluna3", "label" pelos nomes
reais das colunas no seu arquivo

dados_selecionados = dados.select("coluna1", "coluna2", "coluna3",
"label")

# Criar o VectorAssembler para transformar as colunas de entrada
em uma única coluna de features

assembler = VectorAssembler(inputCols=["coluna1", "coluna2",
"coluna3"], outputCol="features")

# Escalar as features usando StandardScaler

scaler = StandardScaler(inputCol="features",
outputCol="scaledFeatures")

# Configurar o modelo de Regressão Linear
```

```
lr = LinearRegression(featuresCol="scaledFeatures",
labelCol="label")

# Criar o Pipeline com as etapas definidas

pipeline = Pipeline(stages=[assembler, scaler, lr])

# Dividir os dados em conjuntos de treino e teste

treino, teste = dados_selecionados.randomSplit([0.8, 0.2],
seed=12345)

# Treinar o Pipeline com os dados de treino

modelo = pipeline.fit(treino)

# Fazer previsões nos dados de teste

previsoes = modelo.transform(teste)

# Avaliar o modelo usando RegressionEvaluator

evaluator = RegressionEvaluator(labelCol="label",
predictionCol="prediction", metricName="rmse")

rmse = evaluator.evaluate(previsoes)

# Exibir o RMSE (Root Mean Squared Error) das previsões

print(f"Root Mean Squared Error (RMSE) no conjunto de teste:
{rmse}")

# Mostrar algumas previsões

previsoes.select("prediction", "label", "features").show(5)
```

```
# Parar a SparkSession
```

```
spark.stop()
```

Conectando à Realidade:

Uma empresa de comércio eletrônico em expansão, a Comércio OnLine Ltda., enfrenta desafios significativos ao analisar os dados de seus milhões de usuários. Com transações diárias gerando uma quantidade imensa de informações, é essencial que a empresa compreenda melhor o comportamento dos clientes para personalizar ofertas, otimizar estoques e aprimorar a experiência do usuário. As ferramentas convencionais de análise de dados não são suficientes para lidar com o volume, velocidade e diversidade dos dados gerados, o que resulta em análises lentas e pouco eficazes.

Caso:

Diante desse cenário, a Comércio OnLine Ltda. optou por adotar PySpark como solução para superar esses obstáculos e elevar sua capacidade analítica. O objetivo é aproveitar a poderosa combinação entre Python e Spark para processar grandes volumes de dados em tempo real e extrair insights valiosos. A empresa tem uma variedade de dados para examinar, incluindo históricos de compras, padrões de navegação no site, interações nas redes sociais e feedback dos clientes. Portanto, eles buscam uma solução que permita integrar todas essas fontes de informação e fornecer análises ágeis e precisas.

Solução:

A Comércio OnLine Ltda. implementou PySpark como ferramenta-chave para processamento e análise dos seus dados. Em primeiro

lugar, um cluster Spark foi configurado para distribuir eficientemente o processamento entre diversos nós.

Em seguida, o PySpark foi usado para ler e combinar dados de várias fontes, como arquivos CSV, JSON e bancos de dados SQL. A equipe de análise de dados realiza consultas complexas usando DataFrames e Spark SQL para identificar padrões comportamentais dos clientes, prever tendências de compra e personalizar recomendações de produtos.

Além disso, a empresa Comércio OnLine Ltda. utiliza a biblioteca MLlib do PySpark para aplicar algoritmos de aprendizado de máquina em seus dados, como clustering para segmentação de clientes e regressão para prever vendas futuras. A implementação automatizada dos pipelines de dados simplifica o processo analítico desde a coleta até a obtenção de insights.

Como resultado, a Comércio OnLine Ltda. consegue processar e analisar seus dados em tempo real, o que possibilita decisões mais embasadas e estratégias de marketing mais eficazes. A personalização das ofertas e a otimização do estoque contribuem significativamente para aumentar a satisfação do cliente e as vendas.

Referências:

APACHE SPARK. **Apache Spark – Unified Engine for large-scale data analytics.**

Disponível em: <https://spark.apache.org/>. Acesso em: 1 jul. 2024.

APACHE SPARK. **PySpark.** Disponível em: <https://spark.apache.org/docs/latest/api/python/>. Acesso em: 9 jul. 2024.

BEHRMAN, Kennedy R. **Fundamentos de Python para ciência de dados.** Grupo A, 2023. E-book. ISBN 9788582605974. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9788582605974/>. Acesso em: 20 maio 2024.

PEREIRA, Mariana A. *et al.* **Framework de Big Data.** Grupo A, 2020. E-book. ISBN 9786556900803. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9786556900803/>. Acesso em: 1 jul. 2024.

Aplicações Avançadas e Visualização de Dados

Autoria: Rodrigo Amaral Rocha

Leitura crítica: Elisa Antolli

Objetivos

- Compreender a análise em tempo real com Spark Streaming.
- Aplicar a capacidade de processamento distribuído do Spark para atividades práticas, como monitoramento de redes sociais, detecção de fraudes e análise de logs em tempo real.
- Aplicar técnicas e ferramentas de visualização de dados: simplificar a representação gráfica de dados complexos utilizando ferramentas como Tableau, Power BI e D3.js. Criar gráficos interativos, mapas e painéis para identificar padrões, tendências e anomalias nos dados.
- Elevar o nível da visualização de dados por meio de ferramentas como Streamlit, Plotly e Dash.
- Explorar os recursos sólidos do Plotly e Dash para desenvolver visualizações complexas e interativas.

1. Para iniciarmos a conversa

O processamento em tempo real é importante para diversas aplicações modernas que exigem respostas instantâneas a eventos contínuos. O Apache Spark Streaming estende a funcionalidade principal do Spark, permitindo o processamento de fluxos de dados em tempo real. Com ele, é possível realizar análises em tempo real de dados que chegam de fontes como Kafka, Flume, sockets TCP e arquivos, entre outros.

2. Apache Spark

De acordo com Pereira (2020), o Apache Spark é uma plataforma robusta de computação em cluster criada para lidar rapidamente com grandes quantidades de dados de forma abrangente. Originado na Universidade da Califórnia, Berkeley, o Spark se destacou como um dos principais projetos da Apache Software Foundation, sendo amplamente utilizado por empresas de diferentes áreas graças à sua eficácia e flexibilidade no processamento de dados.

2.1 Características

Processamento em tempo real: Spark Streaming processa dados em pequenos intervalos de tempo chamados *micro-batches*. Isso permite a análise contínua e quase em tempo real de fluxos de dados, proporcionando insights imediatos e a capacidade de responder rapidamente a eventos. A seguir, estão elencadas mais características desta ferramenta:

- Escalabilidade e tolerância a falhas: herdando a arquitetura escalável e tolerante a falhas do Spark Core, Spark Streaming pode processar grandes volumes de dados em clusters distribuídos.

Ele utiliza o mecanismo de recuperação de falhas do Spark para garantir que os dados sejam processados corretamente, mesmo em caso de falhas.

- API unificada: Spark Streaming integra-se perfeitamente com outras bibliotecas do Spark, como Spark SQL, MLlib e GraphX. Isso permite aos desenvolvedores combinar análises em tempo real com consultas SQL, aprendizado de máquina e processamento de gráficos em uma única aplicação.
- Facilidade de uso: a API de Spark Streaming é simples e intuitiva, permitindo que os desenvolvedores escrevam aplicações de streaming usando as mesmas abstrações e paradigmas de programação do Spark. Isso inclui o uso de DataFrames e Datasets, que facilitam a manipulação e transformação de dados (Apache Spark, 2024).

Além de conhecer as características do Spark Streaming, serão apresentados na sequência os componentes desta ferramenta.

2.2 Componentes

Os componentes do Spark Streaming servem para apoiar ações de transformação de dados em informação, tornando-as essenciais à tomada de decisão. São quatro principais, a saber, descritos nos tópicos que seguem:

- DStreams (*Discretized Streams*): a principal abstração de Spark Streaming é o DStream, que representa um fluxo contínuo de dados dividido em pequenos batches discretos. Os DStreams são criados a partir de fontes de dados (como Kafka, Flume, sockets TCP) e podem ser transformados usando operações semelhantes às aplicadas a RDDs. (Bohnert *et al.*, 2024).

- **Receivers:** componentes que recebem dados de fontes externas e os armazenam na memória do Spark para processamento. Existem diferentes tipos de receivers para suportar várias fontes de dados, incluindo receivers personalizados que podem ser criados para atender a requisitos específicos.
- **Windowed Operations:** Spark Streaming suporta operações de janelas, permitindo que os desenvolvedores realizem cálculos em intervalos de tempo específicos (janelas) sobre os dados de streaming. Isso é útil para análises agregadas e monitoramento de eventos ao longo do tempo.
- **Checkpointing:** para garantir a tolerância a falhas e a recuperação de estado, Spark Streaming suporta o checkpointing de dados e metadados. Os checkpoints podem ser armazenados em sistemas de arquivos distribuídos, como HDFS, para garantir a continuidade do processamento em caso de falhas.

Após entender as características e os componentes do Spark Streaming, é preciso aprender a configurar a ferramenta de forma correta. Observe as recomendações de configurações que seguem.

2.3 Configuração

Para configurar o Spark Streaming, é necessário definir um contexto de streaming que gerencia a execução das operações de streaming. O código a seguir demonstra a configuração básica:

```
from pyspark import SparkContext

from pyspark.streaming import StreamingContext

# Criação do contexto Spark
```

```
sc = SparkContext("local[2]", "NetworkWordCount")

ssc = StreamingContext(sc, 1) # Intervalo de batch de 1 segundo

# Criação do stream a partir de um socket TCP

lines = ssc.socketTextStream("localhost", 9999)

# Processamento do stream

words = lines.flatMap(lambda line: line.split(" "))

wordCounts = words.map(lambda word: (word, 1)).
reduceByKey(lambda a, b: a + b)

# Impressão dos resultados

wordCounts.pprint()

# Início do processamento de streaming

ssc.start()

ssc.awaitTermination()
```

Neste exemplo, o Spark Streaming está configurado para receber dados de um socket TCP, dividir as linhas em palavras, contar as ocorrências de cada palavra e imprimir os resultados a cada segundo.

3. Técnicas e Ferramentas de Visualização de Dados

A visualização de dados é uma etapa essencial na análise de dados, pois permite que percepções complexas sejam apresentadas de maneira

intuitiva e compreensível. Diversas técnicas e ferramentas estão disponíveis para criar visualizações eficazes, cada uma adequada a diferentes tipos de dados e necessidades analíticas.

3.1 Técnicas de Visualização de Dados

Segundo Behrman (2023), a representação visual de dados é uma prática essencial no campo da ciência de dados, responsável por transformar informações brutas em gráficos que facilitam a compreensão e a comunicação de dados complexos. Com o grande volume de informações geradas diariamente, a capacidade de interpretar e visualizar esses dados se tornou uma habilidade relevante para profissionais da área, como cientistas de dados, analistas, desenvolvedores e tomadores de decisão em vários setores.

A visualização de dados se mostra substancial em sua análise por várias razões:

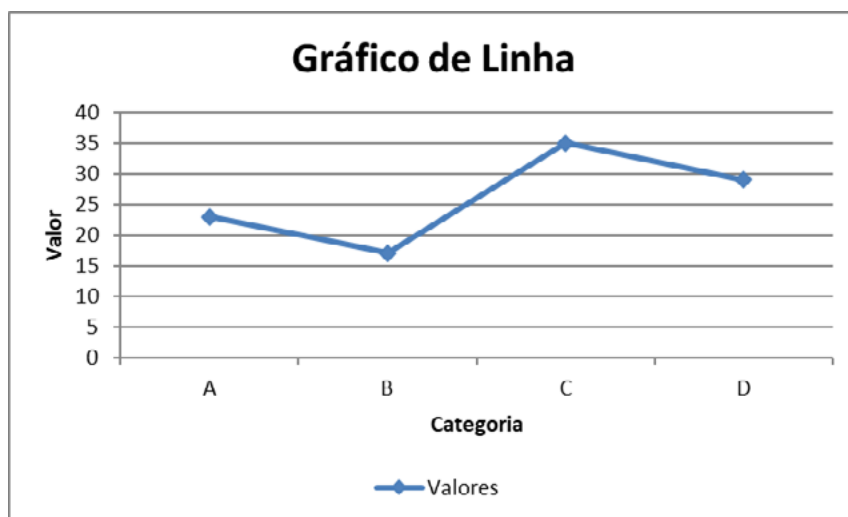
- **Facilitar a compreensão:** gráficos bem elaborados tornam informações complexas mais acessíveis e fáceis de entender, ajudando na identificação de padrões, tendências e discrepâncias que podem passar despercebidas em conjuntos brutos de dados.
- **Auxilia na tomada de decisões:** ao apresentar os dados com clareza e simplicidade, as representações visuais ajudam os tomadores de decisão a compreenderem rapidamente as implicações das informações e tomar decisões embasadas em evidências.
- **Comunicação eficiente:** gráficos atrativos e informativos são ferramentas poderosas para comunicar descobertas e insights para públicos diversos, desde colegas até executivos e clientes.

Existem diversas técnicas e ferramentas para visualizar dados, cada uma adequada para diferentes tipos de análise e conjuntos de dados. Entre as técnicas mais comuns estão:

- Gráficos de linha: ideais para visualizar tendências ao longo do tempo.

Exemplo: Evolução das vendas mensais.

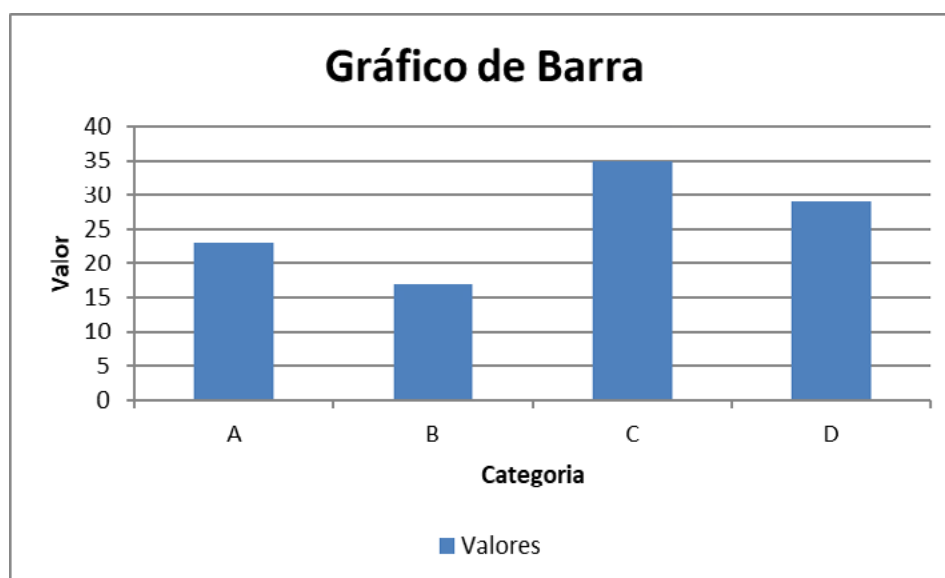
Figura 1 – Exemplo de gráfico de linha



Fonte: elaborado pelo autor.

- Gráficos de barras: usados para comparar valores entre diferentes categorias. Exemplo: vendas por região.

Figura 2 – Exemplo de gráfico de barra



Fonte: elaborado pelo autor.

- Gráficos de pizza: mostram a proporção de partes em relação ao todo. Exemplo: distribuição de participação de mercado.

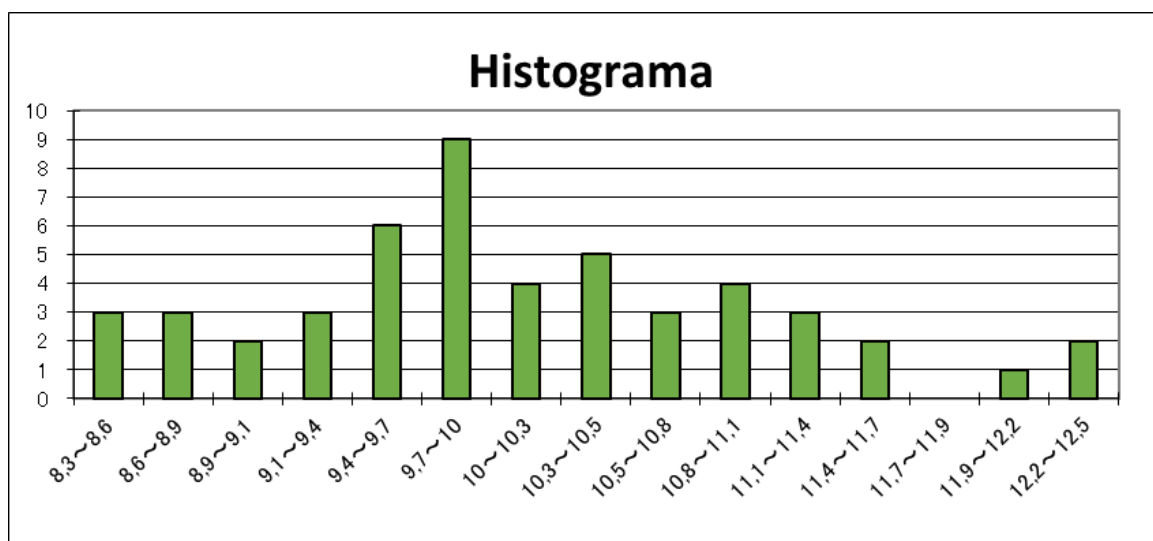
Figura 3 – Exemplo de gráfico de pizza



Fonte: elaborado pelo autor.

- Histogramas: utilizados para visualizar a distribuição de um conjunto de dados. Exemplo: distribuição de idades dos clientes.

Figura 4 – Exemplo de histograma



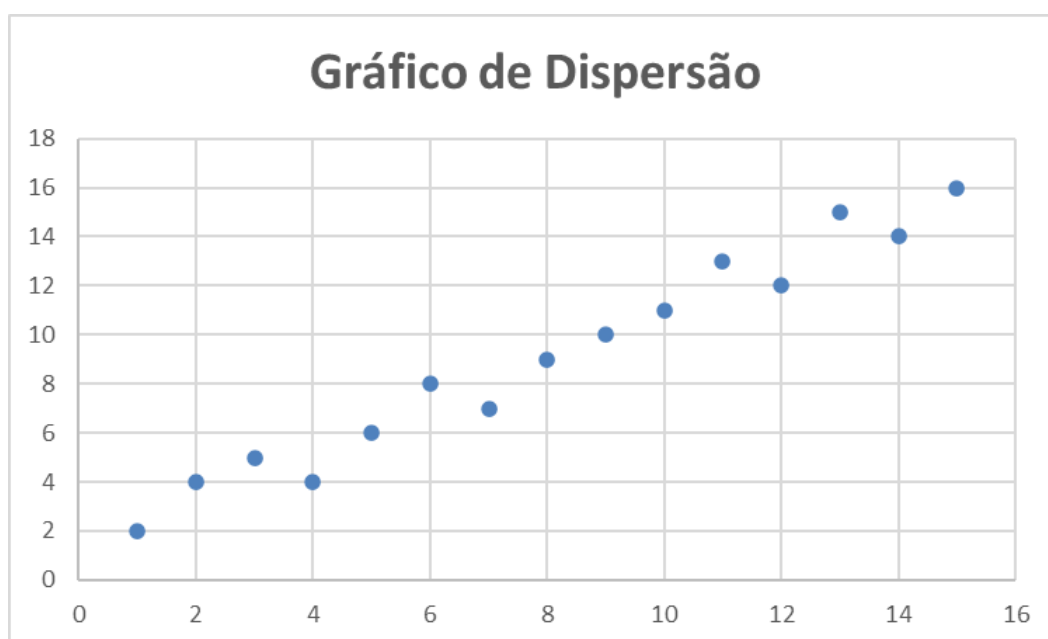
Fonte: elaborado pelo autor.

- Diagramas de dispersão:

Exibem a relação entre duas variáveis contínuas.

Exemplo: Relação entre publicidade e vendas.

Figura 5 – Exemplo de gráfico de dispersão



Fonte: elaborado pelo autor.

3.2 Ferramentas de Visualização de Dados

Na sociedade digital atual, é essencial ter a capacidade de analisar e compreender grandes quantidades de dados para tomar decisões bem fundamentadas. As ferramentas de visualização de dados desempenham um papel relevante nesse processo, transformando informações intrincadas e extensas em representações visuais intuitivas e interativas. Essas ferramentas permitem que profissionais de diversas áreas identifiquem padrões, tendências e discrepâncias de maneira ágil e eficiente, facilitando a comunicação de descobertas e incentivando a colaboração entre equipes (Behrman, 2023).

3.2.1 Importância das Ferramentas

- Facilidade de uso: ferramentas modernas de visualização de dados são projetadas para serem intuitivas e fáceis de usar, permitindo que usuários com diferentes níveis de habilidade técnica criem visualizações significativas sem necessidade de codificação extensa.
- Interatividade: a interatividade é um componente-chave das ferramentas de visualização de dados, permitindo que os usuários explorem os dados em tempo real, façam zoom em detalhes específicos e realizem análises ad-hoc, tudo isso com uma interface visual atraente.
- Integração com fontes de dados diversas: essas ferramentas são capazes de se conectar a uma ampla variedade de fontes de dados, desde bancos de dados SQL e NoSQL até arquivos CSV e APIs da web. Essa flexibilidade facilita a integração dos dados de diferentes sistemas e a criação de visualizações abrangentes.
- Colaboração e compartilhamento: ferramentas de visualização de dados frequentemente incluem recursos de colaboração e compartilhamento, permitindo que equipes trabalhem juntas em dashboards e relatórios, compartilhem insights com facilidade e mantenham todos os stakeholders informados.

3.2.2 Principais Ferramentas

Tableau

De acordo com o site Tableau Software (2024), Tableau é uma das ferramentas de visualização de dados mais populares e poderosas disponíveis. Ele oferece uma interface intuitiva de arrastar e soltar que permite criar visualizações complexas e interativas rapidamente.

- Recursos: suporte para uma ampla gama de fontes de dados, incluindo bases de dados, planilhas e serviços na nuvem. Oferece capacidades avançadas de análise e criação de dashboards interativos.
- Uso: amplamente utilizado em empresas para análise de negócios, criação de relatórios financeiros, monitoramento de KPIs e muito mais.

Power BI

Desenvolvido pela Microsoft, o Power BI é uma ferramenta de visualização de dados que permite a criação de dashboards e relatórios interativos. É especialmente popular em ambientes corporativos devido à sua integração com outros produtos Microsoft.

- Recursos: integração perfeita com Excel, Azure, e outras ferramentas Microsoft. Oferece uma interface amigável e uma ampla variedade de visualizações personalizáveis.
- Uso: utilizado para análise de vendas, monitoramento de desempenho, visualização de dados financeiros e criação de relatórios executivos (Microsoft, 2024).

D3.js

D3.js (*Data-Driven Documents*) é uma biblioteca JavaScript que permite criar visualizações altamente personalizadas e interativas para a web. Oferece controle granular sobre os elementos gráficos e a capacidade de manipular o DOM diretamente.

- Recursos: flexibilidade extrema para criar visualizações únicas. Suporta integração com outros frameworks JavaScript e pode lidar com dados dinâmicos e interativos.

- Uso: ideal para desenvolvedores web que precisam criar visualizações complexas e customizadas, como gráficos animados, mapas interativos e dashboards personalizados (Bohnert *et al.*, 2024).

Plotly

Plotly é uma biblioteca gráfica que suporta a criação de visualizações interativas em várias linguagens de programação, incluindo Python, R e JavaScript. É conhecida por sua facilidade de uso e capacidade de criar visualizações de alta qualidade.

- Recursos: suporte para uma ampla gama de gráficos, desde gráficos simples até visualizações tridimensionais complexas. Oferece integração com ferramentas de análise de dados e notebooks Jupyter.
- Uso: utilizado para visualizações científicas, dashboards de dados em tempo real, análises financeiras e apresentações interativas.

Exemplo:

```
import plotly.express as px

df = px.data.iris()

fig = px.scatter(df, x='sepal_width', y='sepal_length', color='species')

fig.show()
```

Matplotlib e Seaborn

Matplotlib é uma biblioteca Python para criação de gráficos estáticos, animados e interativos. Seaborn, segundo Waskom (2024), é construída sobre o Matplotlib e fornece uma interface de alto nível para criar gráficos estatísticos atraentes.

- Recursos: Matplotlib oferece grande flexibilidade e controle sobre os elementos gráficos, enquanto Seaborn simplifica a criação de gráficos estatísticos com menos código.
- Uso: amplamente utilizado na comunidade científica e acadêmica para visualização de dados experimentais, análise de dados estatísticos e criação de gráficos para publicações.

Exemplo 1:

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4]

y = [10, 20, 25, 30]

plt.plot(x, y)

plt.xlabel('x')

plt.ylabel('y')

plt.title('Exemplo de Gráfico de Linha')

plt.show()
```

Exemplo 2:

```
import seaborn as sns

import pandas as pd

data = pd.DataFrame({

    'x': [1, 2, 3, 4],
```

```
'y': [10, 20, 25, 30]

})

sns.lineplot(data=data, x='x', y='y')

plt.show()
```

3.3 Dashboards Interativos

Dashboards interativos permitem que os usuários explorem e interajam com os dados de maneira dinâmica, oferecendo uma visão mais profunda e personalizada das informações. Ferramentas como Streamlit, Plotly Dash e Dash fornecem plataformas poderosas para a criação de dashboards interativos.

Streamlit

Streamlit é uma biblioteca Python que facilita a criação de aplicativos web interativos para análise de dados (Streamlit, 2024).

1. Instalação:

```
pip install streamlit
```

2. Exemplo de aplicação:

```
import streamlit as st
```

```
import pandas as pd
```

```
st.title('Exemplo de Dashboard com Streamlit')
```

```
df = pd.DataFrame({
```

```
'x': [1, 2, 3, 4],  
'y': [10, 20, 25, 30]  
})
```

```
st.line_chart(df)
```

3. Execução da aplicação:

```
streamlit run nome_do_arquivo.py
```

Plotly Dash

Dash é uma biblioteca baseada no Plotly que permite a criação de aplicativos analíticos interativos com componentes HTML, CSS e JavaScript.

Instalação:

```
pip install dash
```

Exemplo de aplicação:

```
import dash  
  
from dash import dcc, html  
  
import plotly.express as px  
  
import pandas as pd  
  
app = dash.Dash(__name__)  
  
df = pd.DataFrame({
```

```

'x': [1, 2, 3, 4],

'y': [10, 20, 25, 30]

})

fig = px.line(df, x='x', y='y')

app.layout = html.Div(children=[

html.H1(children='Exemplo de Dashboard com Dash'),

dcc.Graph(

id='example-graph',

figure=fig

)

])

if __name__ == '__main__':

app.run_server(debug=True)

```

Aplicação avançada com Dash

Para criar dashboards mais complexos com múltiplos gráficos e interatividade avançada, Dash permite a integração de *callbacks* que atualizam componentes do dashboard em tempo real.

Exemplo de *callback*:

```
import dash
```

```
from dash import dcc, html

from dash.dependencies import Input, Output

import plotly.express as px

import pandas as pd

app = dash.Dash(__name__)

df = pd.DataFrame({

    'x': [1, 2, 3, 4],

    'y': [10, 20, 25, 30]

})

app.layout = html.Div(children=[

    html.H1(children='Dashboard Interativo com Callback'),

    dcc.Input(id='input-number', type='number', value=1),

    dcc.Graph(id='output-graph')

])

@app.callback(

    Output('output-graph', 'figure'),

    [Input('input-number', 'value')]

)
```

```
def update_graph(input_value):  
  
    filtered_df = df[df['x'] <= input_value]  
  
    fig = px.line(filtered_df, x='x', y='y')  
  
    return fig  
  
if __name__ == '__main__':  
  
    app.run_server(debug=True)
```

Conectando à Realidade:

Um grande hospital chamado SaúdePlus está tendo dificuldades para monitorar em tempo real o desempenho de suas operações e a qualidade do atendimento aos pacientes. Diariamente, uma quantidade significativa de dados é coletada, incluindo registros de pacientes, informações de tratamento, feedback dos pacientes, inventário de medicamentos e dados clínicos. No entanto, os gestores estão enfrentando desafios ao acessar e analisar esses dados eficientemente, o que afeta negativamente sua capacidade de tomar decisões rápidas e bem fundamentadas.

Caso:

Os desafios específicos que o SaúdePlus enfrenta incluem:

Monitoramento de ocupação e fluxo de pacientes em tempo real: os gestores precisam de uma visão clara da ocupação dos leitos, tempo de espera no pronto-socorro e fluxo de pacientes nas diferentes unidades para otimizar os recursos e reduzir o tempo de espera.

Gestão de inventário de medicamentos: o hospital precisa garantir que os níveis de estoque de medicamentos sejam otimizados para evitar tanto a falta de medicamentos quanto o excesso, o que pode resultar em custos adicionais ou desabastecimento crítico.

Análise de feedback dos pacientes: SaúdePlus recebe feedback contínuo dos pacientes por meio de várias plataformas (pesquisas de satisfação, redes sociais, formulários on-line), mas falta uma maneira eficiente de consolidar e analisar esses dados para melhorar a qualidade do atendimento.

Solução:

Para resolver esses desafios, o hospital implementa um dashboard interativo usando ferramentas de visualização de dados como Tableau, Power BI e D3.js. A solução é dividida em três partes principais:

Monitoramento de ocupação e fluxo de pacientes:

Dashboard de ocupação: um dashboard que mostra em tempo real a ocupação dos leitos, tempo de espera no pronto-socorro e o fluxo de pacientes nas diferentes unidades do hospital. Este dashboard é construído usando Tableau, que se integra aos sistemas de gerenciamento hospitalar para obter dados em tempo real.

Alertas automatizados: configuração de alertas automatizados que notificam os gestores quando a ocupação atinge níveis críticos ou quando o tempo de espera excede os limites aceitáveis, permitindo ações imediatas para alocar recursos de maneira eficiente.

Gestão de inventário de medicamentos:

Dashboard de inventário: um dashboard criado no Power BI que rastreia os níveis de estoque de medicamentos, consumo diário, previsões de demanda e datas de validade. Este dashboard ajuda a equipe de farmácia a manter o controle sobre os medicamentos e a planejar reabastecimentos de forma proativa.

Integração com sistemas de fornecedores: integração com os sistemas dos fornecedores de medicamentos para facilitar pedidos automáticos quando os níveis de estoque caem abaixo de um certo limite, garantindo que nunca haja falta de medicamentos essenciais.

Análise de feedback dos pacientes:

Dashboard de satisfação do paciente: usando D3.js, o hospital desenvolve um dashboard interativo que consolida o feedback dos pacientes de várias fontes. Este dashboard visualiza as métricas de satisfação, identifica áreas problemáticas e permite uma análise detalhada dos comentários dos pacientes.

Análise de tendências: ferramentas de análise de texto são integradas ao dashboard para identificar tendências e padrões nos feedbacks dos pacientes, ajudando a identificar áreas que precisam de melhorias e a monitorar o impacto das mudanças implementadas.

Referências:

APACHE SPARK. **Apache Spark – Unified Engine for large-scale data analytics**. Disponível em: <https://spark.apache.org/>. Acesso em: 1 jul. 2024.

BEHRMAN, Kennedy R. **Fundamentos de Python para ciência de dados**. Grupo A, 2023. E-book. ISBN 9788582605974. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9788582605974/>. Acesso em: 20 maio 2024.

BOHNERT, M. *et al.* **D3.js – Data-Driven Documents**. Disponível em: <https://d3js.org/>. Acesso em: 9 jul. 2024.

HUNTER, J. D. **Matplotlib: Python plotting**. The Matplotlib Development Team, 2024. Disponível em: <https://matplotlib.org/>. Acesso em: 9 jul. 2024.

MICROSOFT. **Power BI**. Disponível em: <https://powerbi.microsoft.com/>. Acesso em: 9 jul. 2024.

PEREIRA, Mariana A. *et al.* **Framework de Big Data**. Grupo A, 2020. E-book. ISBN 9786556900803. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9786556900803/>. Acesso em: 1 jul. 2024.



STREAMLIT. **Streamlit**: The fastest way to build and share data apps. Disponível em: <https://streamlit.io/>. Acesso em: 23 jul. 2024.

TABLEAU SOFTWARE. **Tableau**. Disponível em: <https://www.tableau.com/>. Acesso em: 9 jul. 2024.

WASKOM, M. *et al.* **Seaborn**: statistical data visualization. Disponível em: <https://seaborn.pydata.org/>. Acesso em: 9 jul. 2024.



Bons estudos!